

Computational perspectives on the classical decision problem

Lecture 4 (Prague 11.08.2026)

Bartosz Jan Bednarczyk `bartek@cs.uni.wroc.pl`

TECHNISCHE UNIVERSITÄT WIEN & UNIwersYTET WROCLAWSKI



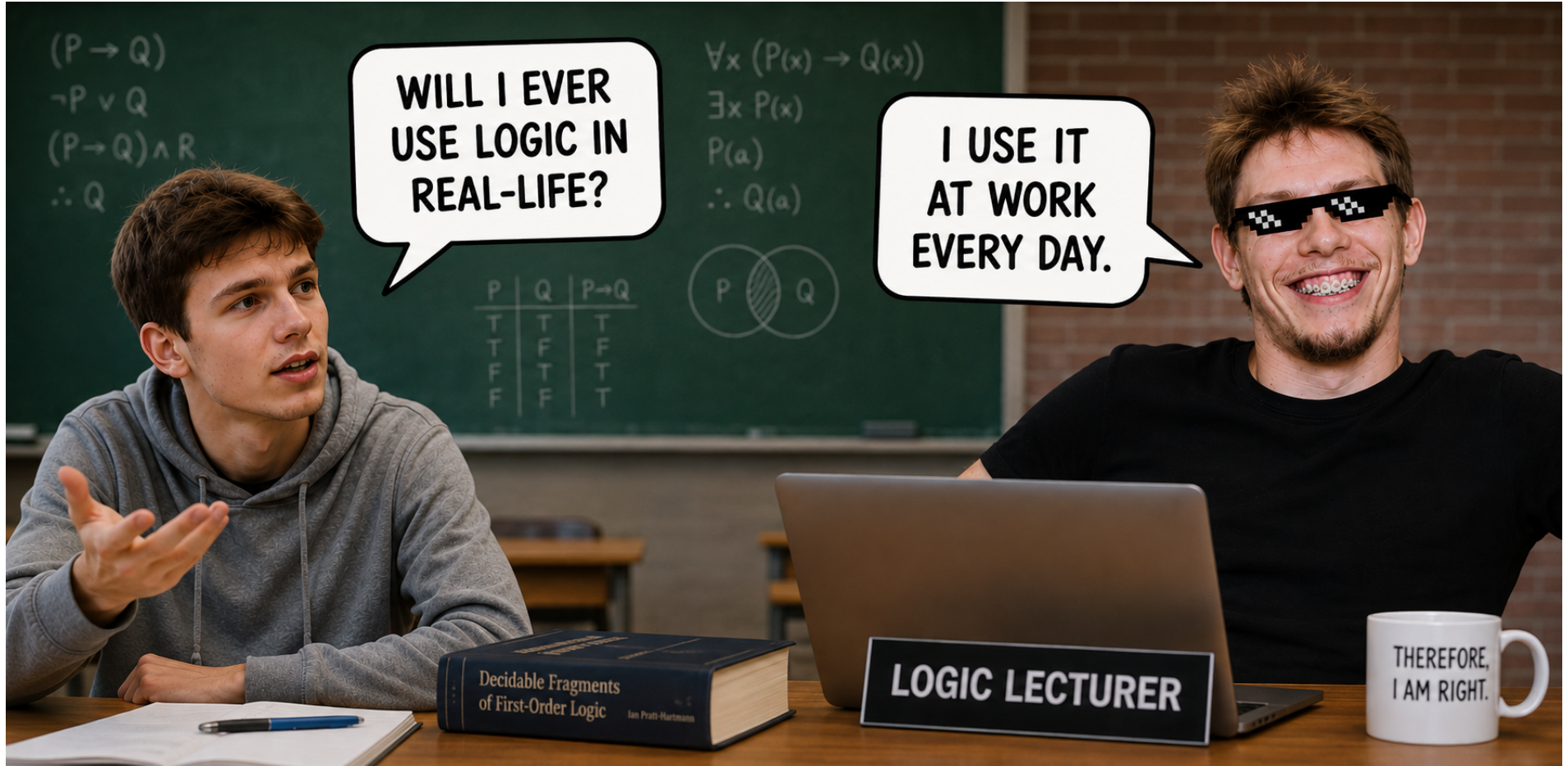
TECHNISCHE
UNIVERSITÄT
WIEN
Vienna | Austria



Uniwersytet
Wrocławski

Useless Complexity Results For Some Random Logics

Lecture 4 (Prague 11.08.2026)



Roadmap: Guards turn global search into local search

Roadmap: Guards turn global search into local search

GF^2 = two-variable first-order logic in which every quantifier is controlled by an atomic guard.

Roadmap: Guards turn global search into local search

GF^2 = two-variable first-order logic in which every quantifier is controlled by an atomic guard.

Goal: $SAT(GF^2) = \text{FinSAT}(GF^2)$ is **ExpTime-complete**.

Roadmap: Guards turn global search into local search

GF^2 = two-variable first-order logic in which every quantifier is controlled by an atomic guard.

Goal: $SAT(GF^2) = \text{FinSAT}(GF^2)$ is **ExpTime-complete**.

1. Define **guarded quantification** and see what it forbids.

Roadmap: Guards turn global search into local search

GF^2 = two-variable first-order logic in which every quantifier is controlled by an atomic guard.

Goal: $SAT(GF^2) = \text{FinSAT}(GF^2)$ is **ExpTime-complete**.

1. Define **guarded quantification** and see what it forbids.
2. Use guards to **obtain locality** and **tree-like models**.

Roadmap: Guards turn global search into local search

GF^2 = two-variable first-order logic in which every quantifier is controlled by an atomic guard.

Goal: $SAT(GF^2) = FinSAT(GF^2)$ is **ExpTime-complete**.

1. Define **guarded quantification** and see what it forbids.
2. Use guards to **obtain locality** and **tree-like models**.
3. **Unravel** a cyclic model into a **guarded tree**.

Roadmap: Guards turn global search into local search

GF^2 = two-variable first-order logic in which every quantifier is controlled by an atomic guard.

Goal: $SAT(GF^2) = FinSAT(GF^2)$ is **ExpTime-complete**.

1. Define **guarded quantification** and see what it forbids.
2. Use guards to **obtain locality** and **tree-like models**.
3. **Unravel** a cyclic model into a **guarded tree**.
4. Search the tree-like models using types, a depth counter, and alternation.

Roadmap: Guards turn global search into local search

GF^2 = two-variable first-order logic in which every quantifier is controlled by an atomic guard.

Goal: $SAT(GF^2) = FinSAT(GF^2)$ is **ExpTime-complete**.

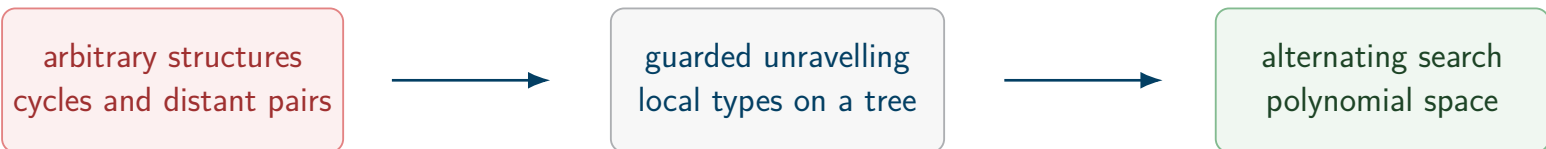
1. Define **guarded quantification** and see what it forbids.
2. Use guards to **obtain locality** and **tree-like models**.
3. **Unravel** a cyclic model into a **guarded tree**.
4. Search the tree-like models using types, a depth counter, and alternation.
5. Combine the upper bound with the finite model property and hardness.

Roadmap: Guards turn global search into local search

GF^2 = two-variable first-order logic in which every quantifier is controlled by an atomic guard.

Goal: $SAT(GF^2) = FinSAT(GF^2)$ is **ExpTime-complete**.

1. Define **guarded quantification** and see what it forbids.
2. Use guards to **obtain locality** and **tree-like models**.
3. **Unravel** a cyclic model into a **guarded tree**.
4. Search the tree-like models using types, a depth counter, and alternation.
5. Combine the upper bound with the finite model property and hardness.

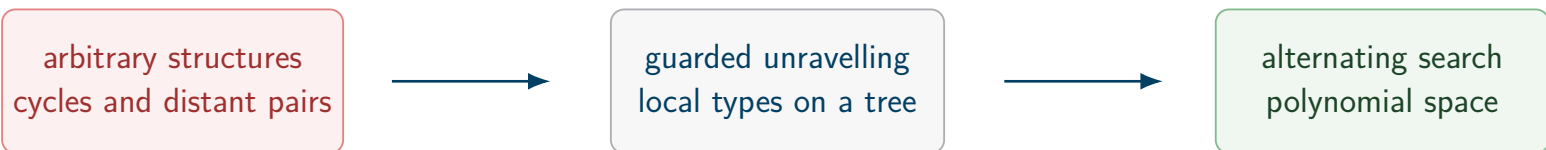


Roadmap: Guards turn global search into local search

GF^2 = two-variable first-order logic in which every quantifier is controlled by an atomic guard.

Goal: $SAT(GF^2) = \text{FinSAT}(GF^2)$ is **ExpTime-complete**.

1. Define **guarded quantification** and see what it forbids.
2. Use guards to **obtain locality** and **tree-like models**.
3. **Unravel** a cyclic model into a **guarded tree**.
4. Search the tree-like models using types, a depth counter, and alternation.
5. Combine the upper bound with the finite model property and hardness.



Feel free to ask questions and interrupt me!

Please don't be shy! If needed, send me an email (bartek@cs.uni.wroc.pl) or approach me before/after the lecture!

If you would like to do a research project on related topics, let me know!

Icon downloaded from icon-icons.com by ©VisualPharm (under CC BY 4.0). No changes have been made.

The guarded fragment restricts quantification patterns

The guarded fragment restricts quantification patterns

Definition (GF (Andréka, van Benthem, and Németi))

The guarded fragment restricts quantification patterns

Definition (GF (Andréka, van Benthem, and Németi))

Atoms, Boolean combinations, and the guarded quantifiers below.

The guarded fragment restricts quantification patterns

Definition (GF (Andréka, van Benthem, and Németi))

Atoms, Boolean combinations, and the guarded quantifiers below.

If $\varphi(\bar{x}, \bar{y}) \in \text{GF}$ and the atom $\gamma(\bar{x}, \bar{y})$ mentions **all** free variables, then

The guarded fragment restricts quantification patterns

Definition (GF (Andréka, van Benthem, and Németi))

Atoms, Boolean combinations, and the guarded quantifiers below.

If $\varphi(\bar{x}, \bar{y}) \in \text{GF}$ and the atom $\gamma(\bar{x}, \bar{y})$ mentions **all** free variables, then

$$\forall \bar{y} \left(\gamma(\bar{x}, \bar{y}) \rightarrow \varphi(\bar{x}, \bar{y}) \right) \quad \text{and} \quad \exists \bar{y} \left(\gamma(\bar{x}, \bar{y}) \wedge \varphi(\bar{x}, \bar{y}) \right)$$

The guarded fragment restricts quantification patterns

Definition (GF (Andréka, van Benthem, and Németi))

Atoms, Boolean combinations, and the guarded quantifiers below.

If $\varphi(\bar{x}, \bar{y}) \in \text{GF}$ and the atom $\gamma(\bar{x}, \bar{y})$ mentions **all** free variables, then

$$\forall \bar{y} \left(\gamma(\bar{x}, \bar{y}) \rightarrow \varphi(\bar{x}, \bar{y}) \right) \quad \text{and} \quad \exists \bar{y} \left(\gamma(\bar{x}, \bar{y}) \wedge \varphi(\bar{x}, \bar{y}) \right)$$

The atom γ is the **guard**: it certifies that the variables form one local tuple.

The guarded fragment restricts quantification patterns

Definition (GF (Andréka, van Benthem, and Németi))

Atoms, Boolean combinations, and the guarded quantifiers below.

If $\varphi(\bar{x}, \bar{y}) \in \text{GF}$ and the atom $\gamma(\bar{x}, \bar{y})$ mentions **all** free variables, then

$$\forall \bar{y} (\gamma(\bar{x}, \bar{y}) \rightarrow \varphi(\bar{x}, \bar{y})) \quad \text{and} \quad \exists \bar{y} (\gamma(\bar{x}, \bar{y}) \wedge \varphi(\bar{x}, \bar{y}))$$

The atom γ is the **guard**: it certifies that the variables form one local tuple.

GUARDED

- $\forall x \forall y (R_{xy} \rightarrow R_{yx})$
- $\forall x (P_x \rightarrow \exists y (R_{xy} \wedge Q_y))$
- $\forall x (C_x \rightarrow D_x)$
- $\exists x (x \approx x \wedge P_x)$

Every quantified tuple is covered by an atom.

NOT GUARDED

- $\forall x \forall y R_{xy}$
- $\forall xyz (R_{xy} \wedge R_{yz} \rightarrow R_{xz})$
- $\forall x \forall y (P_x \wedge P_y \rightarrow x \approx y)$
- $\exists x (P_x \wedge \forall yz (R_{xy} \rightarrow R_{xz}))$

Some quantifier has no atom covering its variables.

The guarded fragment restricts quantification patterns

Definition (GF (Andréka, van Benthem, and Németi))

Atoms, Boolean combinations, and the guarded quantifiers below.

If $\varphi(\bar{x}, \bar{y}) \in \text{GF}$ and the atom $\gamma(\bar{x}, \bar{y})$ mentions **all** free variables, then

$$\forall \bar{y} (\gamma(\bar{x}, \bar{y}) \rightarrow \varphi(\bar{x}, \bar{y})) \quad \text{and} \quad \exists \bar{y} (\gamma(\bar{x}, \bar{y}) \wedge \varphi(\bar{x}, \bar{y}))$$

The atom γ is the **guard**: it certifies that the variables form one local tuple.

GUARDED

- $\forall x \forall y (R_{xy} \rightarrow R_{yx})$
- $\forall x (P_x \rightarrow \exists y (R_{xy} \wedge Q_y))$
- $\forall x (C_x \rightarrow D_x)$
- $\exists x (x \approx x \wedge P_x)$

Every quantified tuple is covered by an atom.

NOT GUARDED

- $\forall x \forall y R_{xy}$
- $\forall xyz (R_{xy} \wedge R_{yz} \rightarrow R_{xz})$
- $\forall x \forall y (P_x \wedge P_y \rightarrow x \approx y)$
- $\exists x (P_x \wedge \forall yz (R_{xy} \rightarrow R_{xz}))$

Some quantifier has no atom covering its variables.

For this talk we focus on: GF^2 the guarded fragment using at most two variables x, y .

Why guards help: they enforce locality!

Why guards help: they enforce locality!

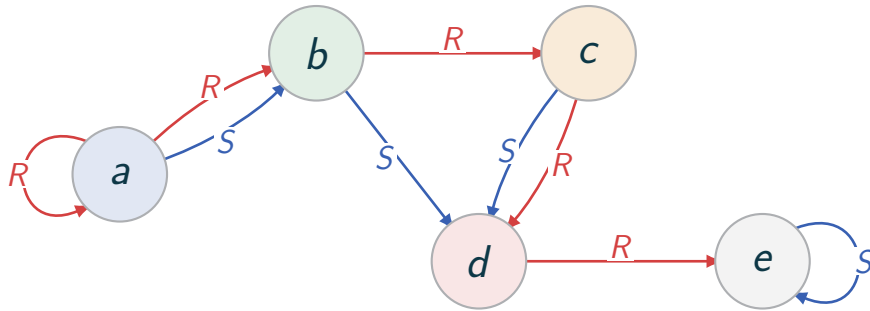
The **Gaifman graph** connects distinct elements whenever they occur together in an atom.

Why guards help: they enforce locality!

The **Gaifman graph** connects distinct elements whenever they occur together in an atom.

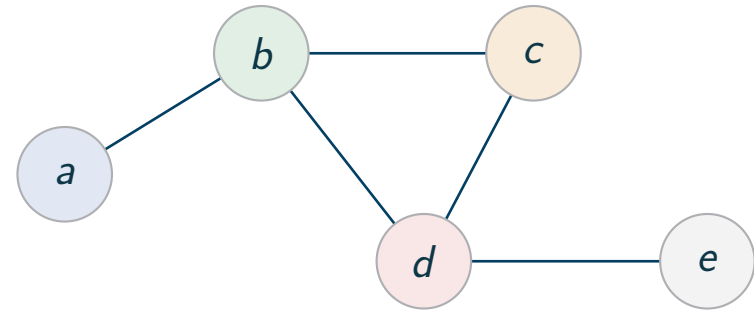
RELATIONAL STRUCTURE $\mathfrak{A}$

R and S are directed binary relations



keep only
co-occurrence
→

GAIFMAN GRAPH $G(\mathfrak{A})$

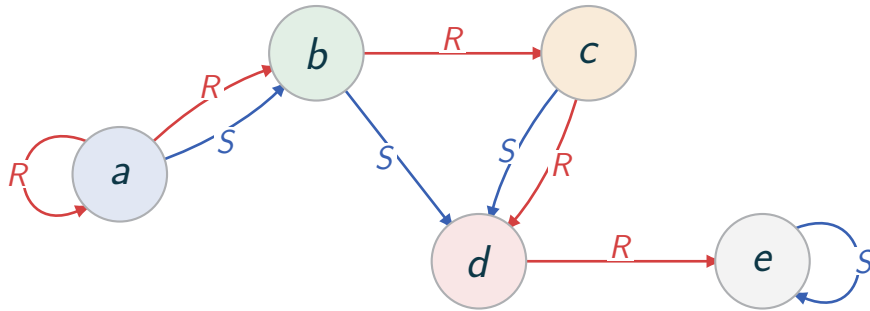


Why guards help: they enforce locality!

The **Gaifman graph** connects distinct elements whenever they occur together in an atom.

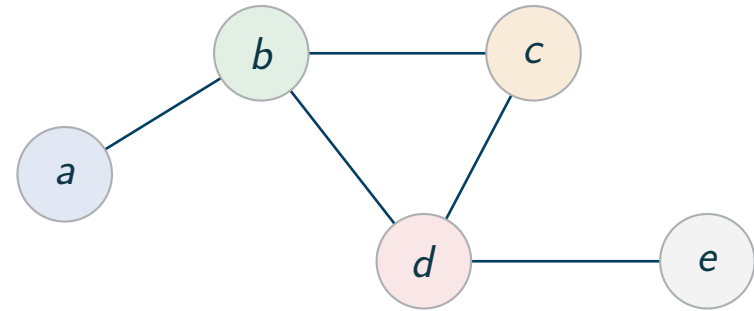
RELATIONAL STRUCTURE $\mathfrak{A}$

R and S are directed binary relations



keep only
co-occurrence
→

GAIFMAN GRAPH $G(\mathfrak{A})$



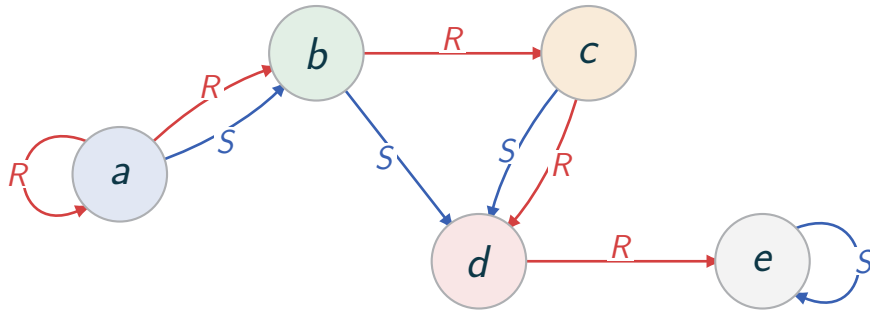
Parallel facts collapse to one edge; self-loops add no Gaifman edge.

Why guards help: they enforce locality!

The **Gaifman graph** connects distinct elements whenever they occur together in an atom.

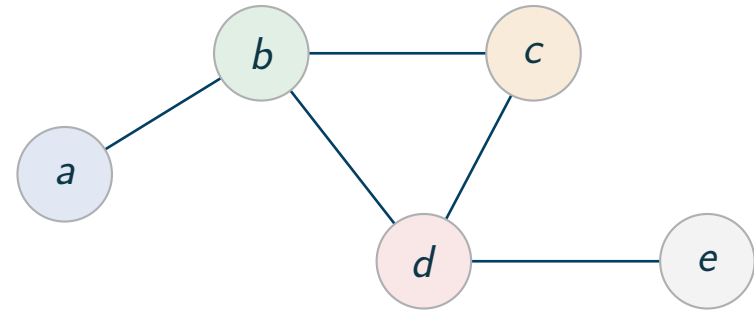
RELATIONAL STRUCTURE $\mathfrak{A}$

R and S are directed binary relations



keep only
co-occurrence
→

GAIFMAN GRAPH $G(\mathfrak{A})$

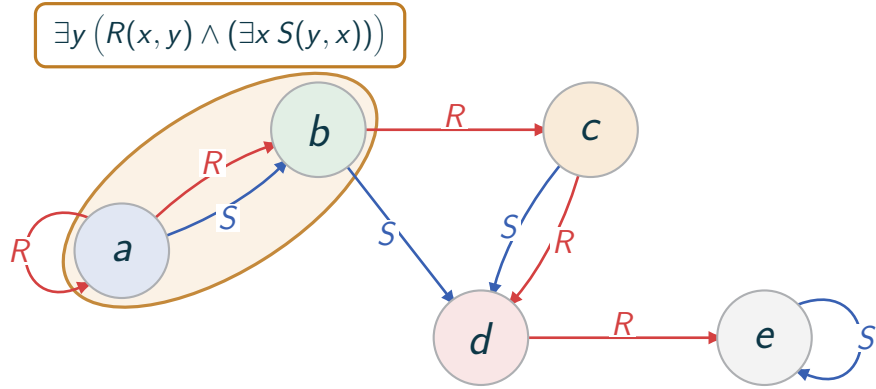


Parallel facts collapse to one edge; self-loops add no Gaifman edge.

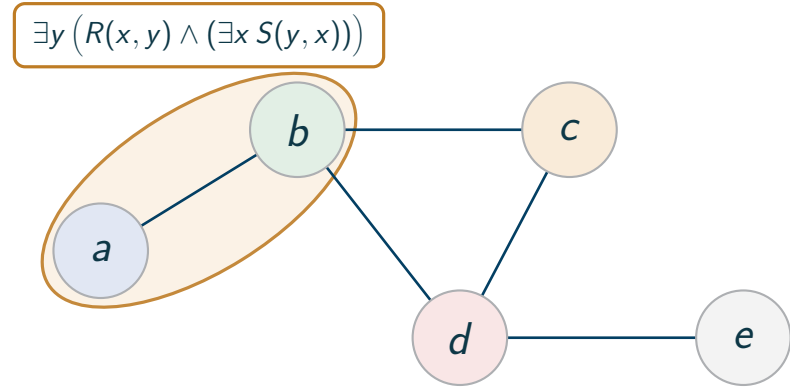
A guarded quantifier can move only inside one such **local neighbourhood**.

Example: An evaluation of $\exists y (R(x, y) \wedge (\exists x S(y, x)))$

RELATIONAL STRUCTURE $\mathfrak{A}$

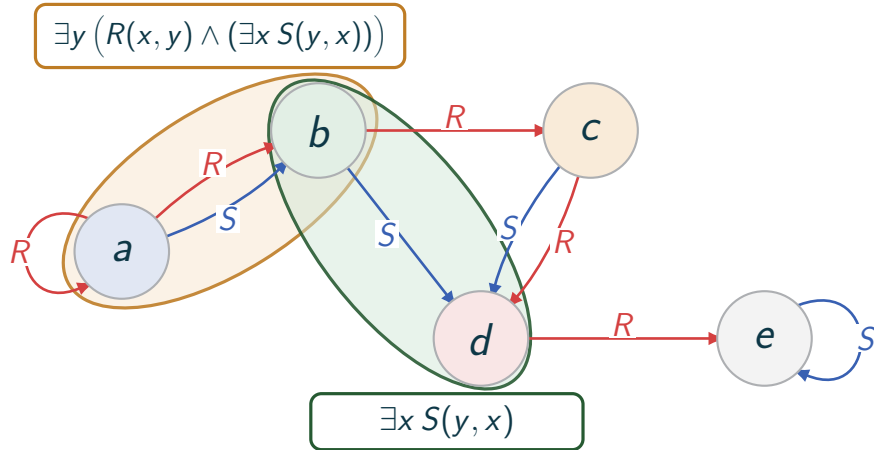
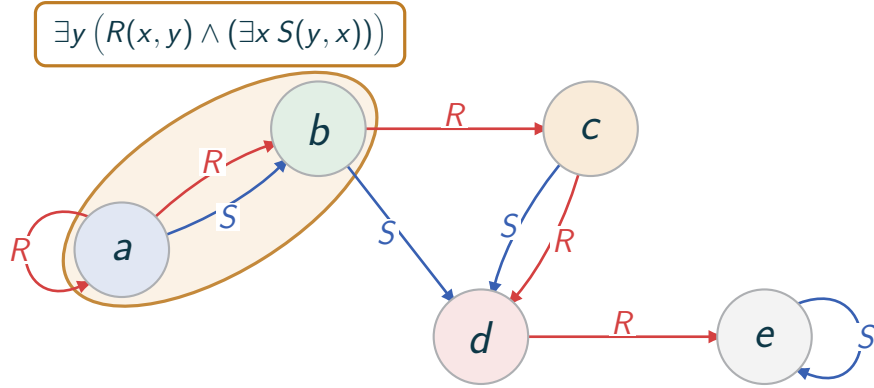


GAIFMAN GRAPH $G(\mathfrak{A})$

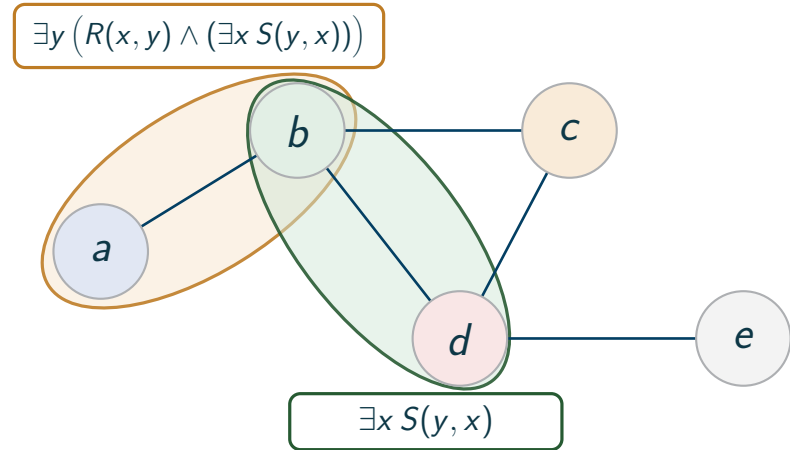
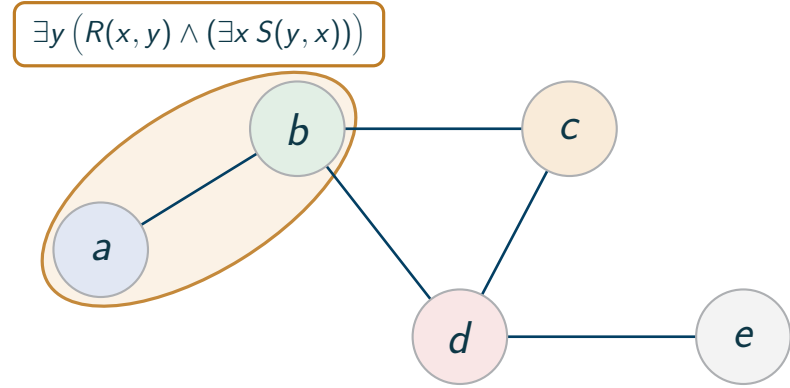


Example: An evaluation of $\exists y (R(x, y) \wedge (\exists x S(y, x)))$

RELATIONAL STRUCTURE $\mathfrak{A}$

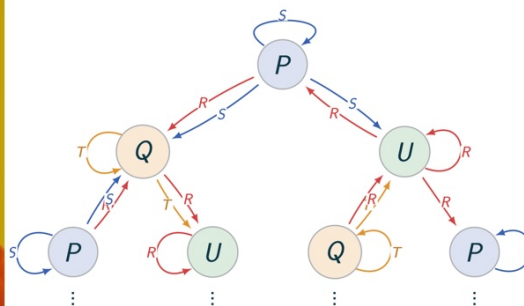
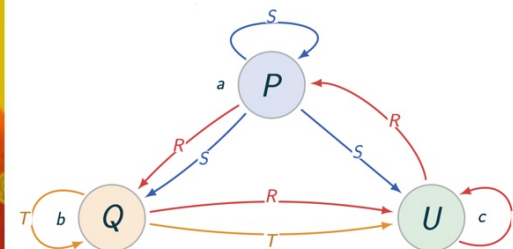
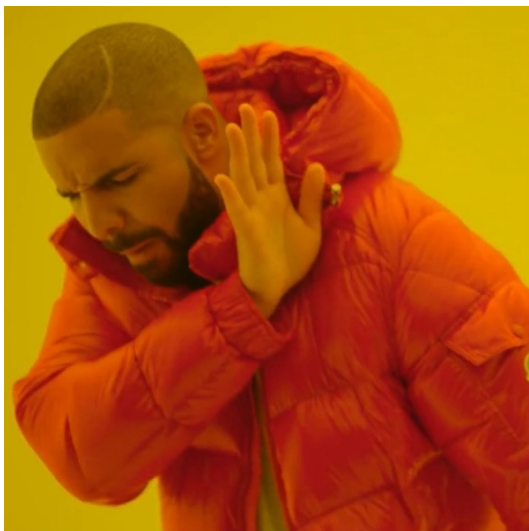


GAIFMAN GRAPH $G(\mathfrak{A})$



From cyclic models to tree-shaped models

From cyclic models to tree-shaped models



From cyclic models to tree-shaped models

From cyclic models to tree-shaped models

Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

From cyclic models to tree-shaped models

Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

From cyclic models to tree-shaped models

Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**.

From cyclic models to tree-shaped models

Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

From cyclic models to tree-shaped models

Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$

From cyclic models to tree-shaped models

Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$

cyclic model $\mathfrak{A}$

tree-shaped unravelling $\mathfrak{B}$

From cyclic models to tree-shaped models

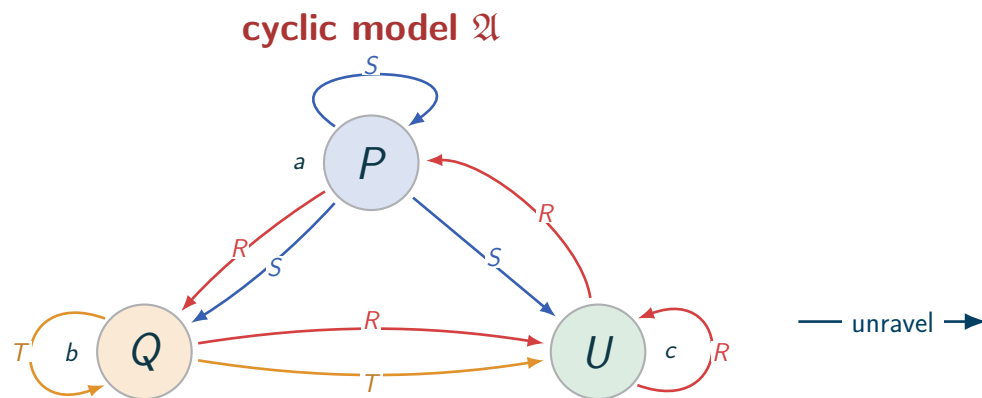
Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$



From cyclic models to tree-shaped models

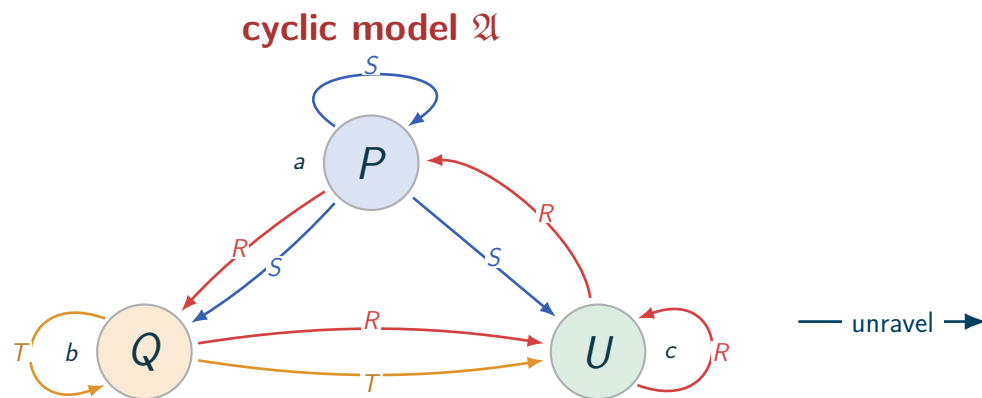
Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$



From cyclic models to tree-shaped models

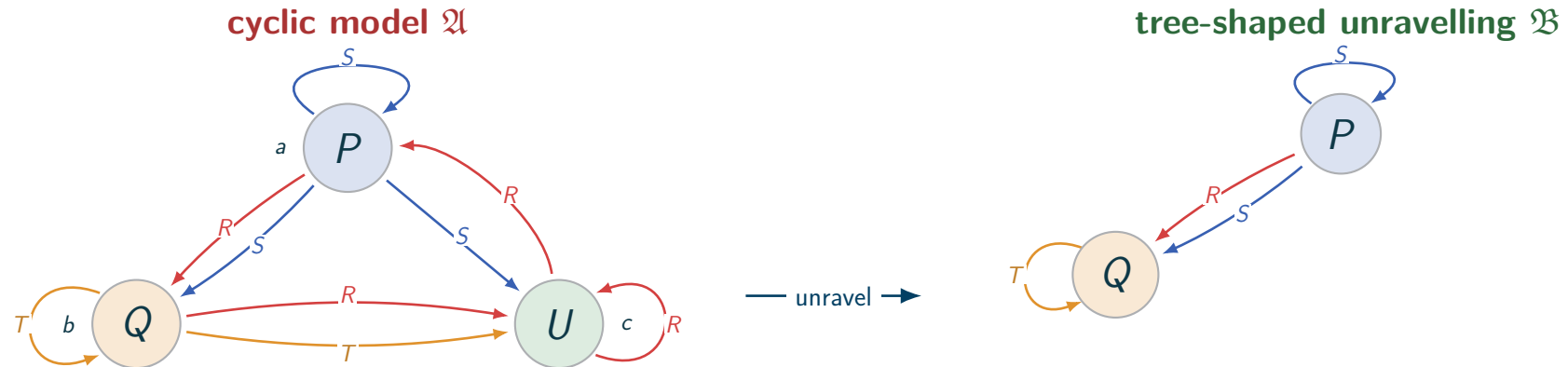
Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$



From cyclic models to tree-shaped models

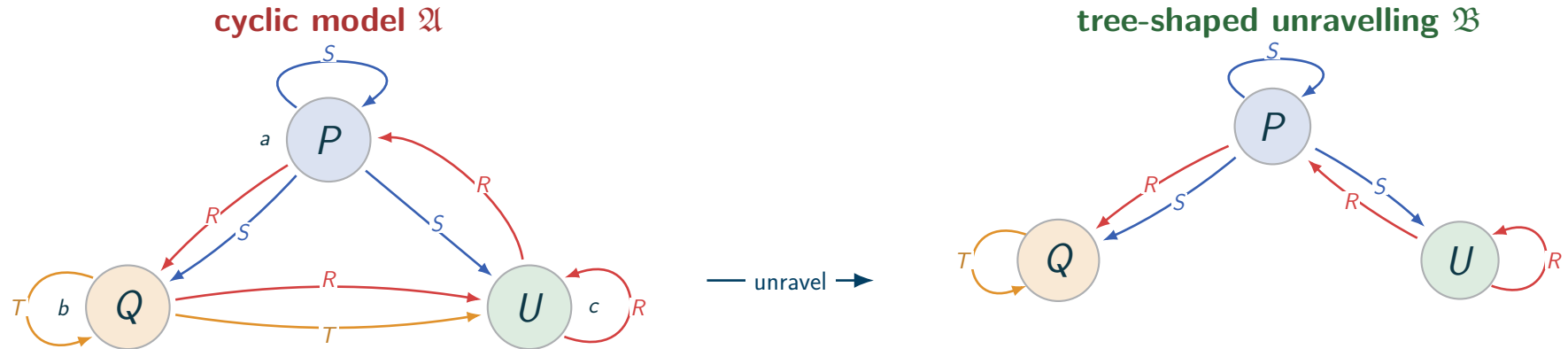
Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$



From cyclic models to tree-shaped models

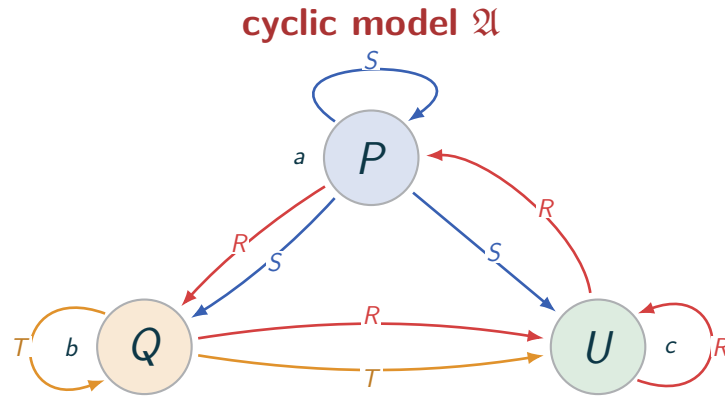
Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

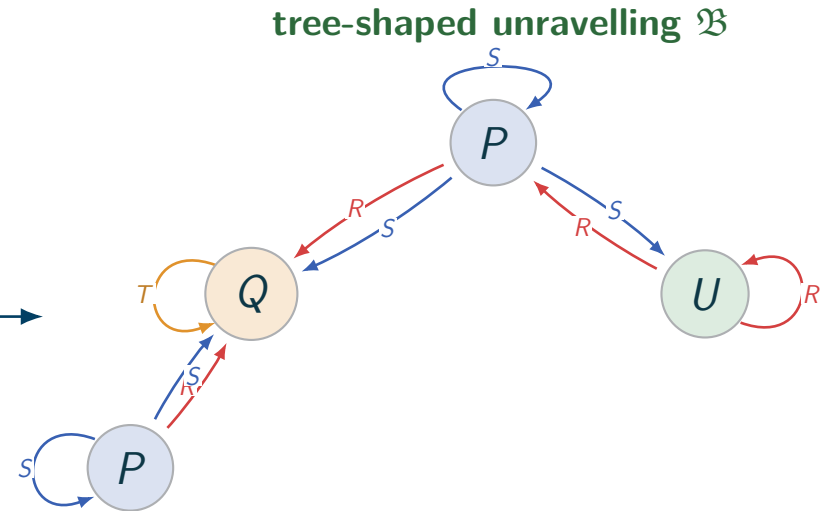
When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$



— unravel →



From cyclic models to tree-shaped models

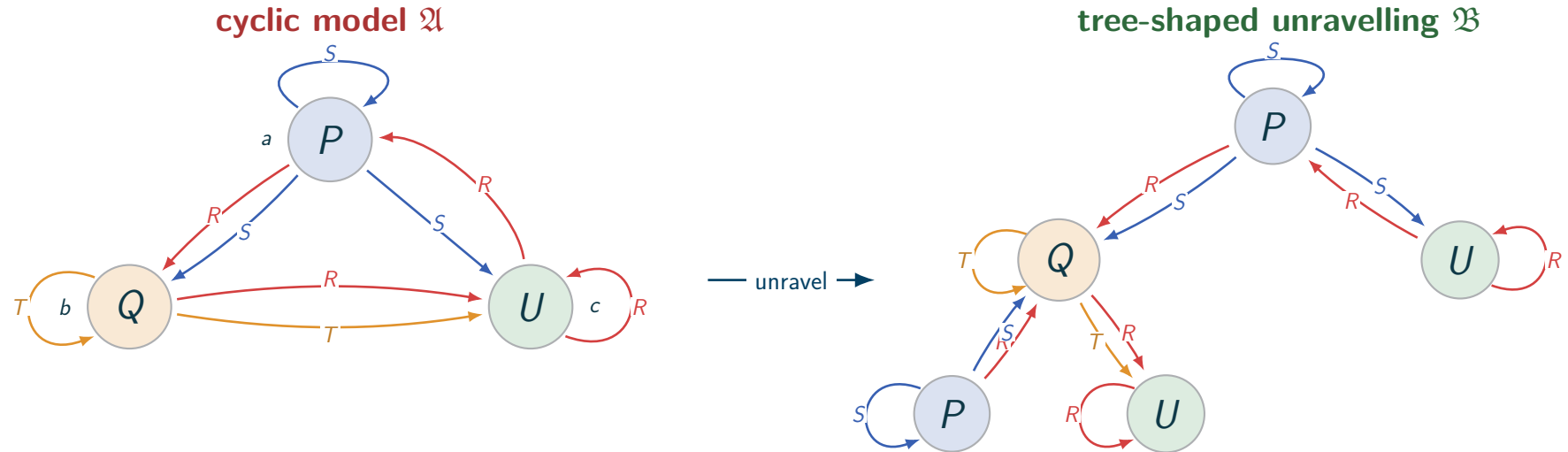
Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$



From cyclic models to tree-shaped models

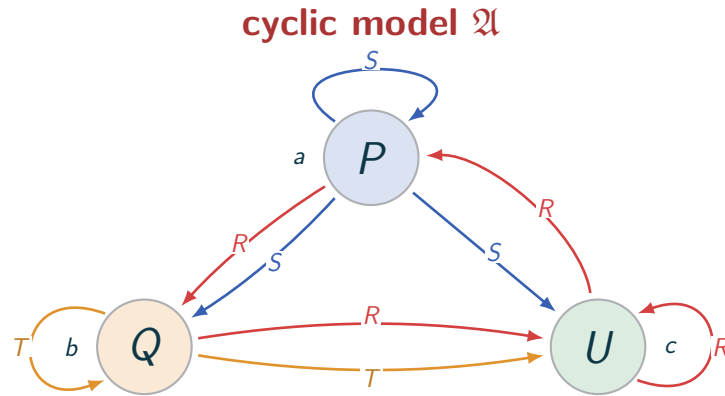
Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

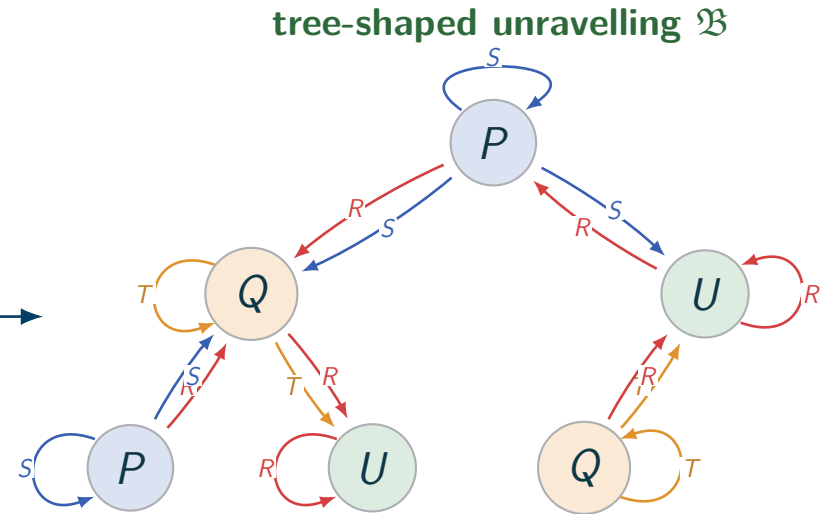
When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$



— unravel →



From cyclic models to tree-shaped models

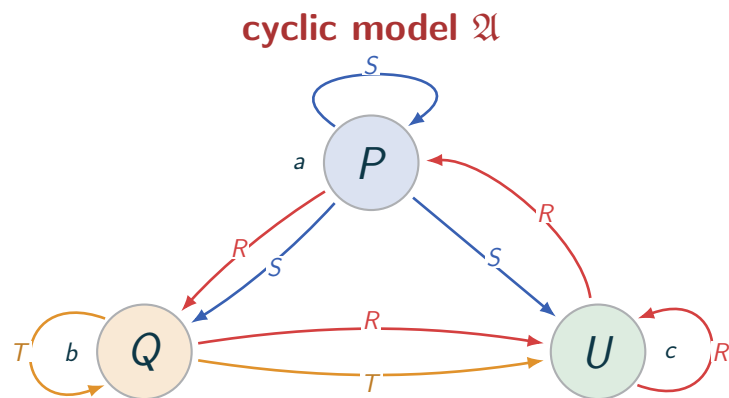
Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

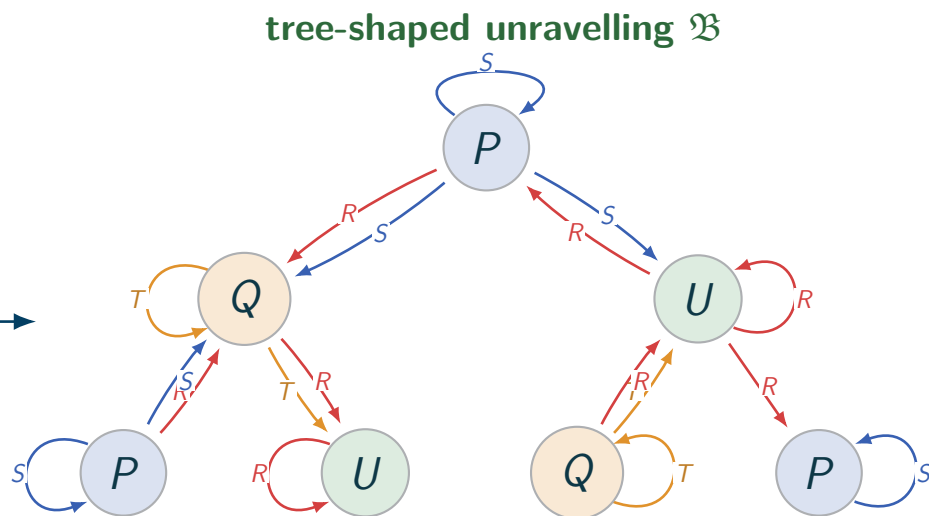
When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$



— unravel →



From cyclic models to tree-shaped models

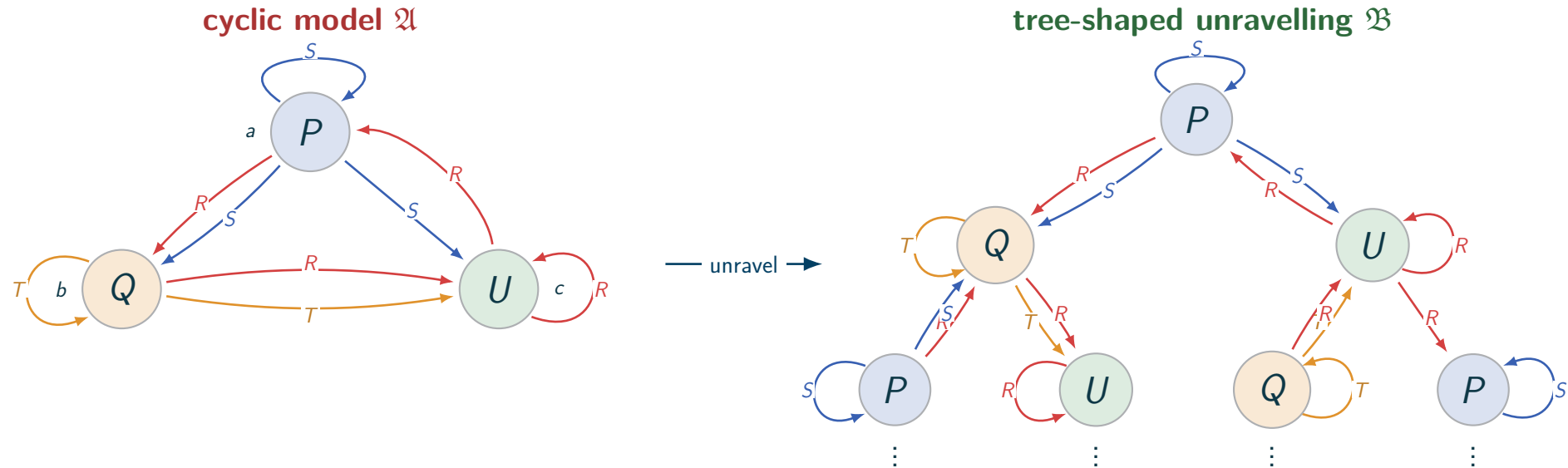
Definition (Tree-shaped model)

$\mathfrak{A}$ is **tree-shaped** if its Gaifman graph is a tree.

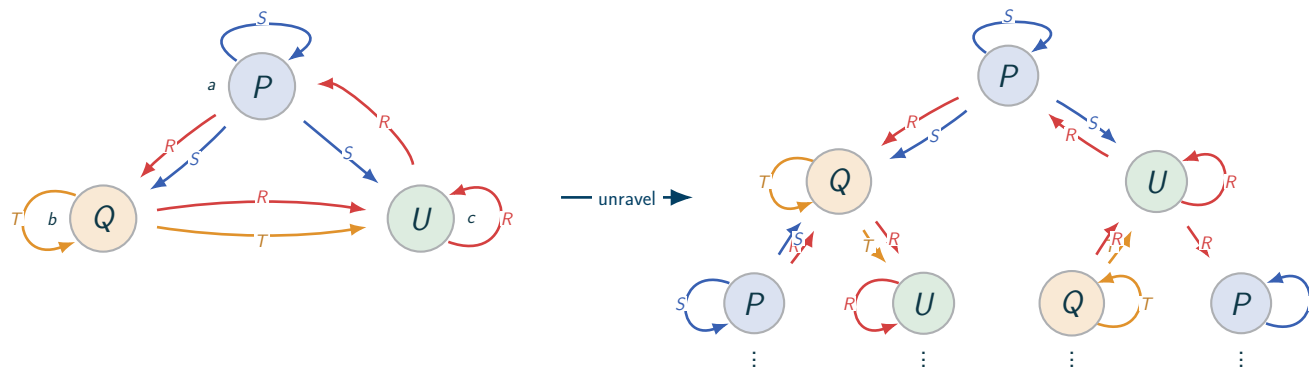
When reasoning in GF^2 , we can **always restrict** our models to the **tree-shaped ones**.

Main tool: **Unravelling**. Construction by example.

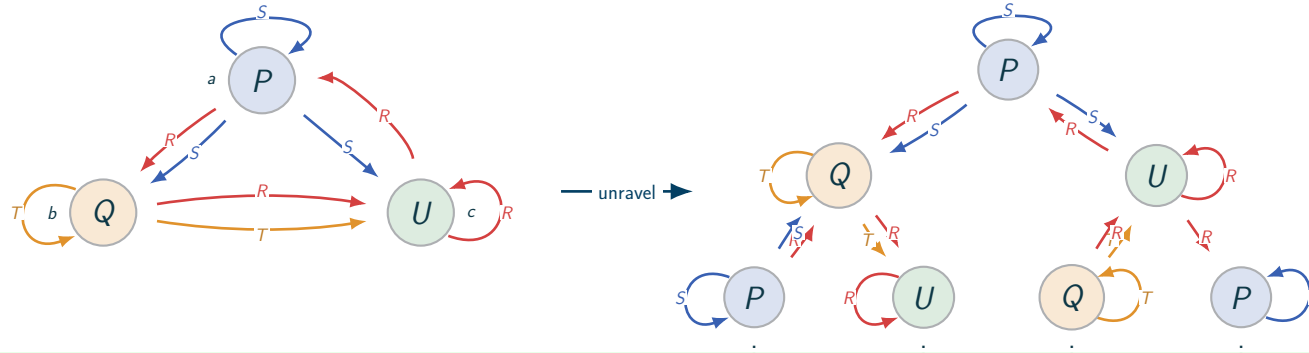
$$\forall x(Px \rightarrow \exists y(Rxy \wedge Qy)) \wedge \forall x(Px \rightarrow \exists y(Sxy \wedge Uy)) \wedge \forall x(Qx \rightarrow \exists y(Txy \wedge Uy)) \wedge \forall x(Ux \rightarrow \exists y(Rxy \wedge Py))$$



Unravelling: the formal construction

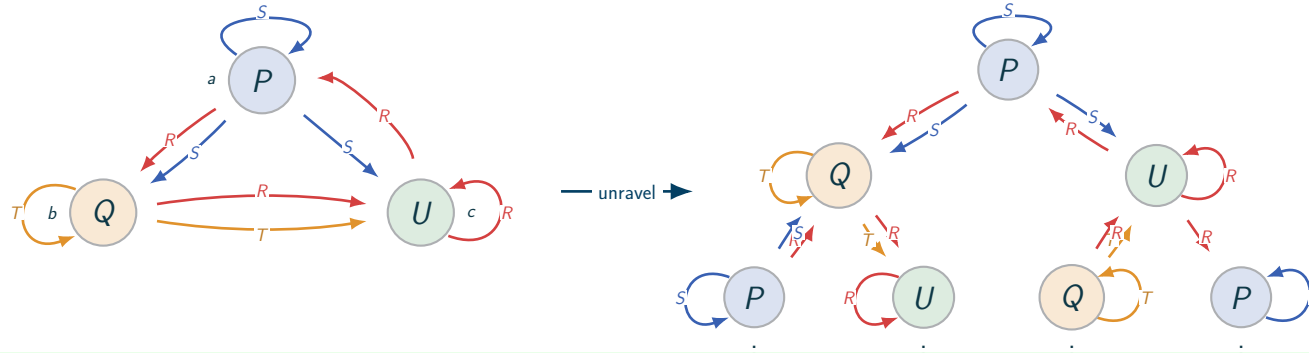


Unravelling: the formal construction



Theorem: Every satisfiable GF^2 formula has a tree-shaped model.

Unravelling: the formal construction



Theorem: Every satisfiable GF^2 formula has a tree-shaped model.

choose a guarded
successor in $\mathcal{Q}$

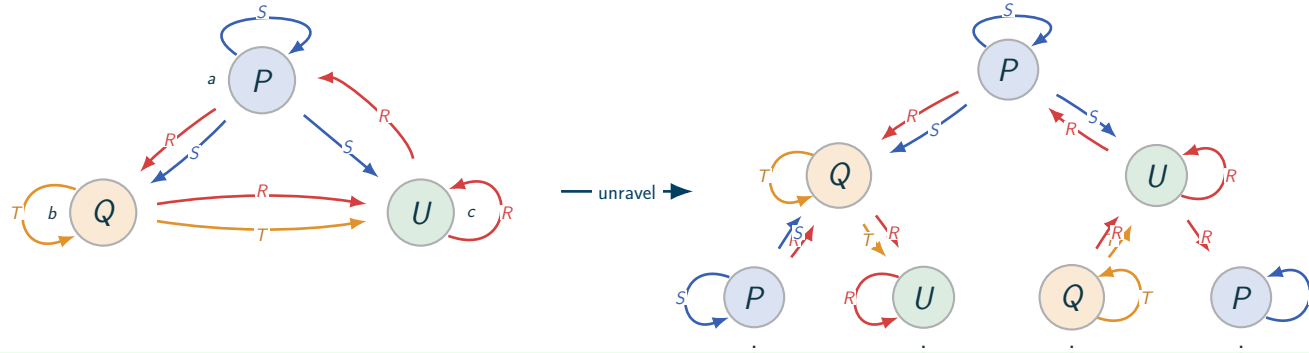


append it to
the current path



copy the local
1- and 2-types

Unravelling: the formal construction



Theorem: Every satisfiable GF^2 formula has a tree-shaped model.

choose a guarded
successor in $\mathfrak{A}$



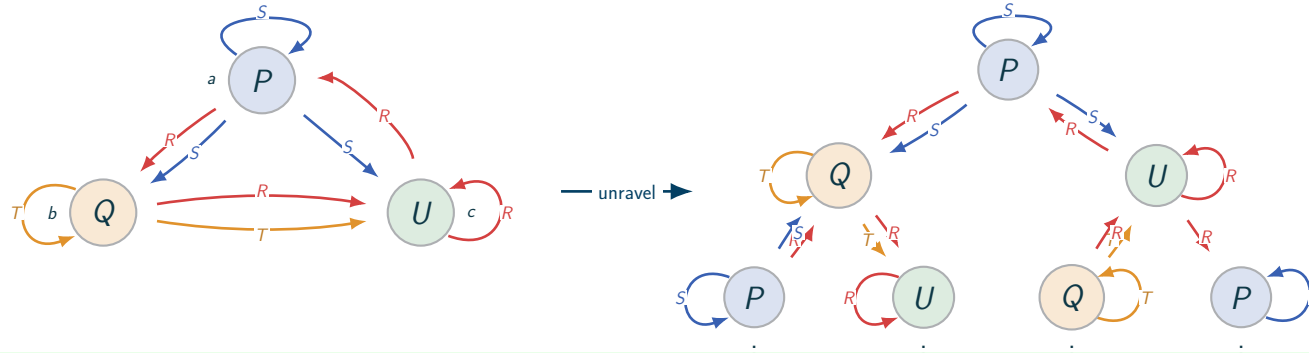
append it to
the current path



copy the local
1- and 2-types

Fix a model $\mathfrak{A}$ and a start element a_0 .

Unravelling: the formal construction



Theorem: Every satisfiable GF^2 formula has a tree-shaped model.

choose a guarded
successor in $\mathfrak{A}$

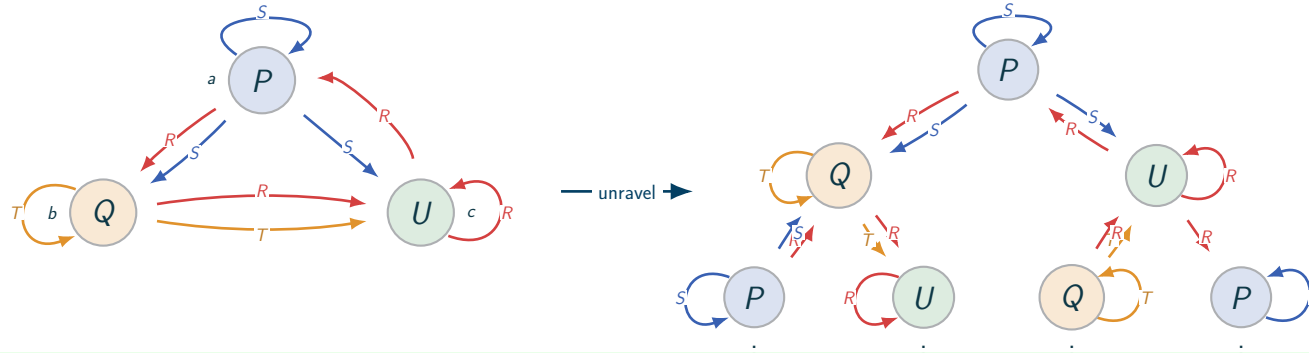
append it to
the current path

copy the local
1- and 2-types

Fix a model $\mathfrak{A}$ and a start element a_0 .

$a_0, a_1, \dots, a_n$ with consecutive a_i, a_{i+1} sharing a guard in $\mathfrak{A}$.

Unravelling: the formal construction



Theorem: Every satisfiable GF^2 formula has a tree-shaped model.

choose a guarded
successor in $\mathfrak{A}$



append it to
the current path



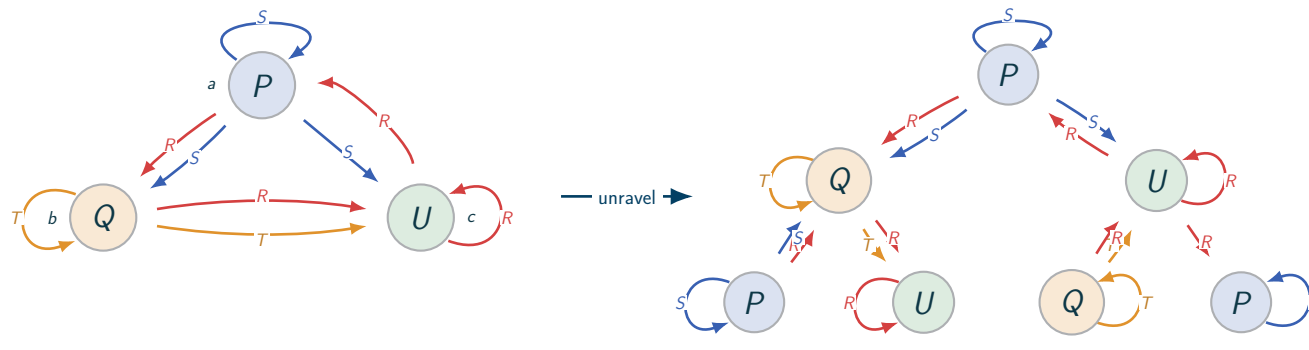
copy the local
1- and 2-types

Fix a model $\mathfrak{A}$ and a start element a_0 .

$a_0, a_1, \dots, a_n$ with consecutive a_i, a_{i+1} sharing a guard in $\mathfrak{A}$.

1. A node of $\mathfrak{B}$ is one such guarded path.

Unravelling: the formal construction



Theorem: Every satisfiable GF^2 formula has a tree-shaped model.

choose a guarded
successor in $\mathfrak{A}$

append it to
the current path

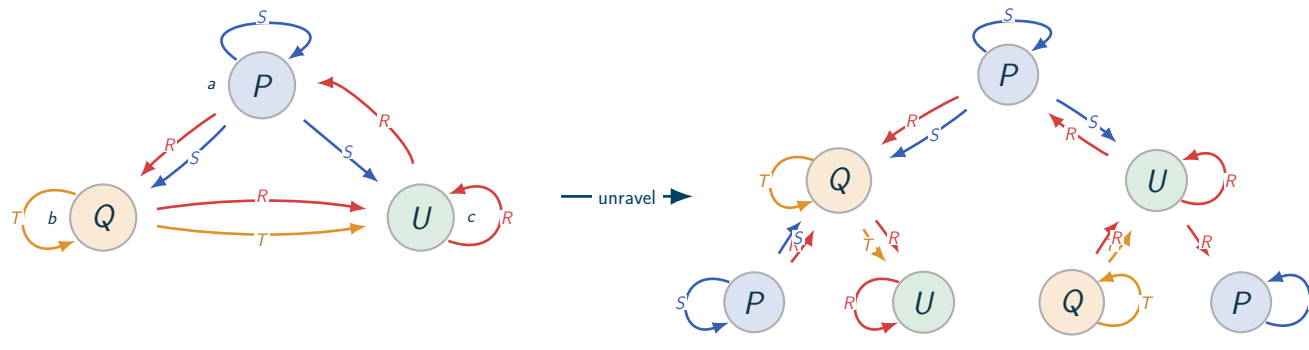
copy the local
1- and 2-types

Fix a model $\mathfrak{A}$ and a start element a_0 .

$a_0, a_1, \dots, a_n$ with consecutive a_i, a_{i+1} sharing a guard in $\mathfrak{A}$.

1. A node of $\mathfrak{B}$ is one such guarded path.
2. Its 1-type is the 1-type of the endpoint a_n .

Unravelling: the formal construction



Theorem: Every satisfiable GF^2 formula has a tree-shaped model.

choose a guarded
successor in $\mathfrak{A}$

append it to
the current path

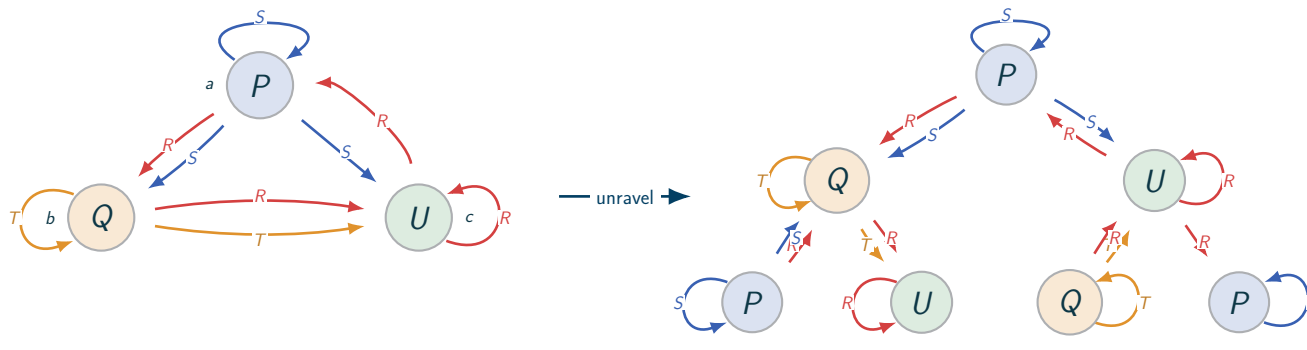
copy the local
1- and 2-types

Fix a model $\mathfrak{A}$ and a start element a_0 .

$a_0, a_1, \dots, a_n$ with consecutive a_i, a_{i+1} sharing a guard in $\mathfrak{A}$.

1. A node of $\mathfrak{B}$ is one such guarded path.
2. Its 1-type is the 1-type of the endpoint a_n .
3. A one-step extension inherits the guarded 2-type of (a_n, a_{n+1}) .

Unravelling: the formal construction



Theorem: Every satisfiable GF^2 formula has a tree-shaped model.

choose a guarded
successor in $\mathfrak{A}$

append it to
the current path

copy the local
1- and 2-types

Fix a model $\mathfrak{A}$ and a start element a_0 .

$a_0, a_1, \dots, a_n$ with consecutive a_i, a_{i+1} sharing a guard in $\mathfrak{A}$.

1. A node of $\mathfrak{B}$ is one such guarded path.
2. Its 1-type is the 1-type of the endpoint a_n .
3. A one-step extension inherits the guarded 2-type of (a_n, a_{n+1}) .

Only a path and its one-step extension share a guard, so the Gaifman graph is a tree.

The tree-model property simplifies the search

The tree-model property simplifies the search

- Suppose $\varphi := \forall x \forall y \chi \wedge \bigwedge_{i=1}^m \forall x (\alpha_i(x) \rightarrow \exists y (\beta_i(x, y) \wedge \psi_i(x, y)))$ in GF^2 is given.

The tree-model property simplifies the search

- Suppose $\varphi := \forall x \forall y \chi \wedge \bigwedge_{i=1}^m \forall x (\alpha_i(x) \rightarrow \exists y (\beta_i(x, y) \wedge \psi_i(x, y)))$ in GF^2 is given.
- By the tree-model property, it suffices to search for a model $\mathfrak{A}$ of φ that is a tree.

The tree-model property simplifies the search

- Suppose $\varphi := \forall x \forall y \chi \wedge \bigwedge_{i=1}^m \forall x (\alpha_i(x) \rightarrow \exists y (\beta_i(x, y) \wedge \psi_i(x, y)))$ in GF^2 is given.
- By the tree-model property, it suffices to search for a model $\mathfrak{A}$ of φ that is a tree.
- It is easy to prove that nodes in the intended model have degree bounded by m .

The tree-model property simplifies the search

- Suppose $\varphi := \forall x \forall y \chi \wedge \bigwedge_{i=1}^m \forall x (\alpha_i(x) \rightarrow \exists y (\beta_i(x, y) \wedge \psi_i(x, y)))$ in GF^2 is given.
- By the tree-model property, it suffices to search for a model $\mathfrak{A}$ of φ that is a tree.
- It is easy to prove that nodes in the intended model have degree bounded by m .

Problem 1: depth

The unravelling **may be infinite**.

We cannot construct or store it **explicitly**.

The tree-model property simplifies the search

- Suppose $\varphi := \forall x \forall y \chi \wedge \bigwedge_{i=1}^m \forall x (\alpha_i(x) \rightarrow \exists y (\beta_i(x, y) \wedge \psi_i(x, y)))$ in GF^2 is given.
- By the tree-model property, it suffices to search for a model $\mathfrak{A}$ of φ that is a tree.
- It is easy to prove that nodes in the intended model have degree bounded by m .

Problem 1: depth

The unravelling may be infinite.

We cannot construct or store it explicitly.

Problem 2: branching

Many witness obligations branch at once.

Too much for backtracking.

The tree-model property simplifies the search

- Suppose $\varphi := \forall x \forall y \chi \wedge \bigwedge_{i=1}^m \forall x (\alpha_i(x) \rightarrow \exists y (\beta_i(x, y) \wedge \psi_i(x, y)))$ in GF^2 is given.
- By the tree-model property, it suffices to search for a model $\mathfrak{A}$ of φ that is a tree.
- It is easy to prove that nodes in the intended model have degree bounded by m .

Problem 1: depth

The unravelling **may be infinite**.

We cannot construct or store it **explicitly**.

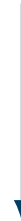


bound depth by types

Problem 2: branching

Many witness obligations branch at once.

Too much for **backtracking**.



replace backtracking by alternation

Fix 1: repeated types bound the depth

Fix 1: repeated types bound the depth

Along any branch, every element carries its 1-type.

Fix 1: repeated types bound the depth

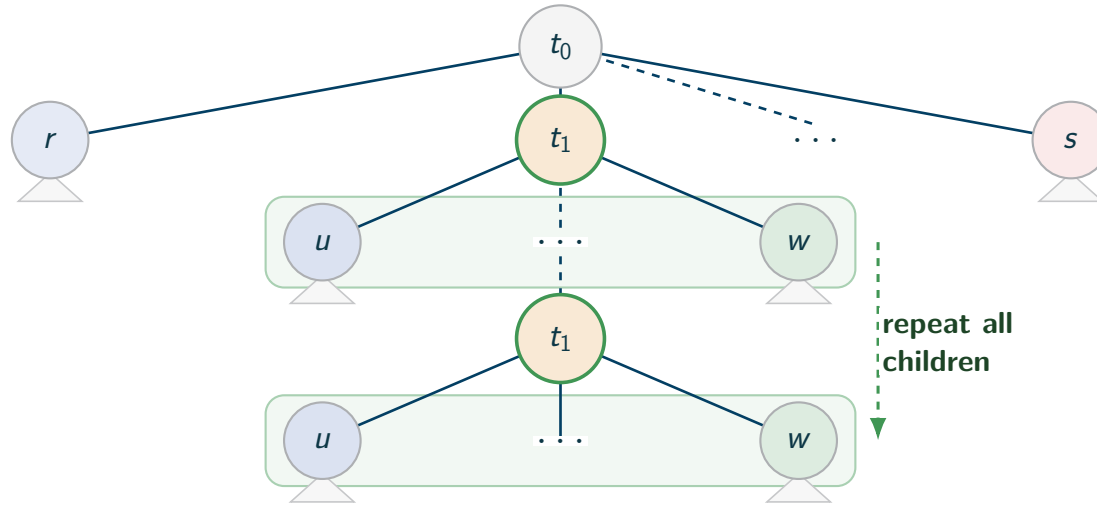
Along any branch, every element **carries its 1-type**.

If a 1-type repeats, the later subtree **may reuse** the earlier local behaviour.

Fix 1: repeated types bound the depth

Along any branch, every element **carries its 1-type**.

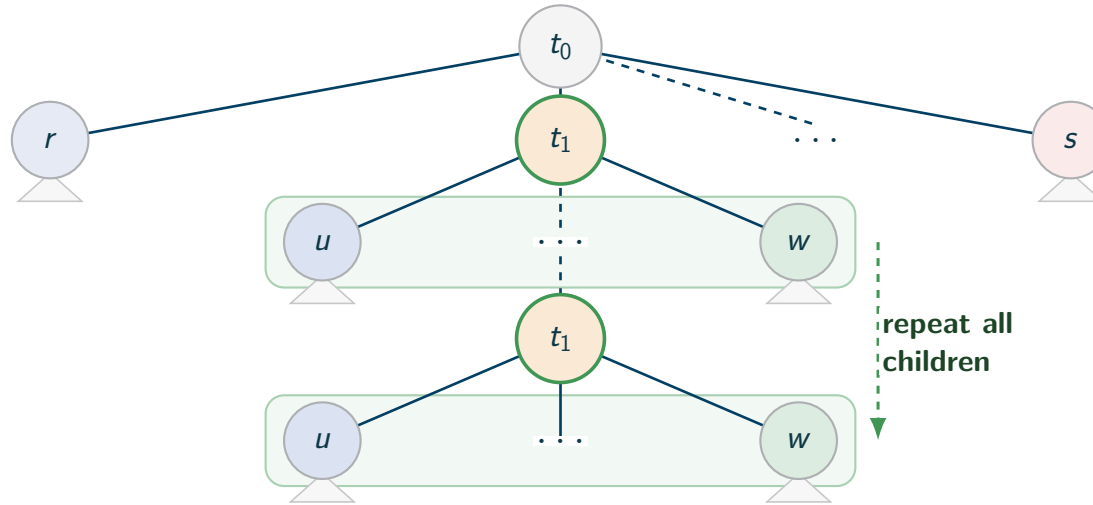
If a 1-type repeats, the later subtree **may reuse** the earlier local behaviour.



Fix 1: repeated types bound the depth

Along any branch, every element **carries its 1-type**.

If a 1-type repeats, the later subtree **may reuse** the earlier local behaviour.

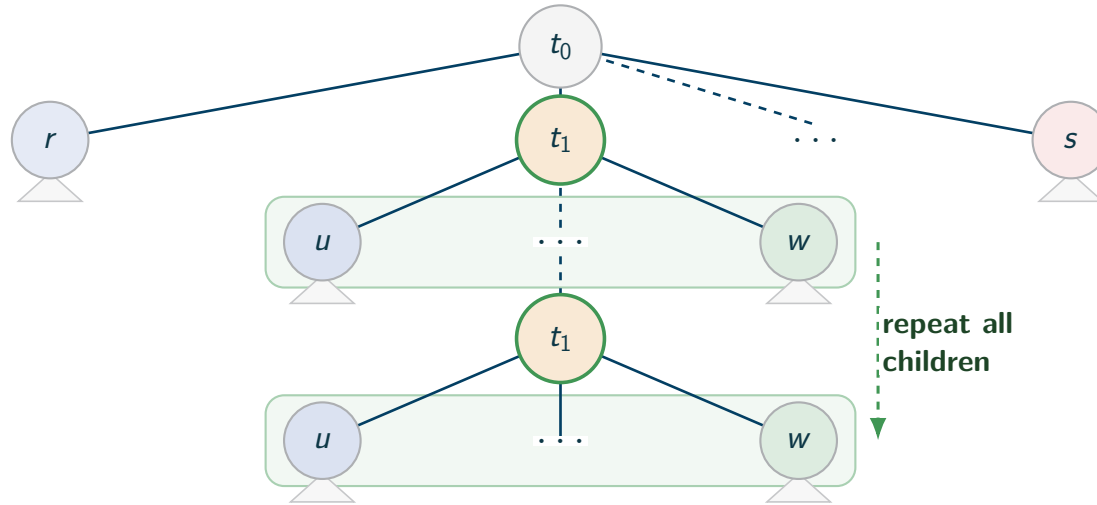


If N is the number of 1-types, then $N \leq 2^{|\varphi|}$.

Fix 1: repeated types bound the depth

Along any branch, every element **carries its 1-type**.

If a 1-type repeats, the later subtree **may reuse** the earlier local behaviour.



If N is the number of 1-types, then $N \leq 2^{|\varphi|}$.

It suffices to explore branches of depth at most N . Why?

Good news: a binary counter for N uses only polynomially many bits!

Fix 2: alternation replaces backtracking

Fix 2: alternation replaces backtracking

EXISTENTIAL STATE

nondeterminism = “guess a solution”

choose one locally consistent type or witness

accept if *some* choice works

Fix 2: alternation replaces backtracking

EXISTENTIAL STATE

nondeterminism = “guess a solution”

choose one locally consistent type or witness

accept if *some* choice works

UNIVERSAL STATE

alternation = “run many threads in parallel”

one thread per triggered witness obligation

accept if *all* threads work

Fix 2: alternation replaces backtracking

EXISTENTIAL STATE

nondeterminism = “guess a solution”

choose one locally consistent type or witness

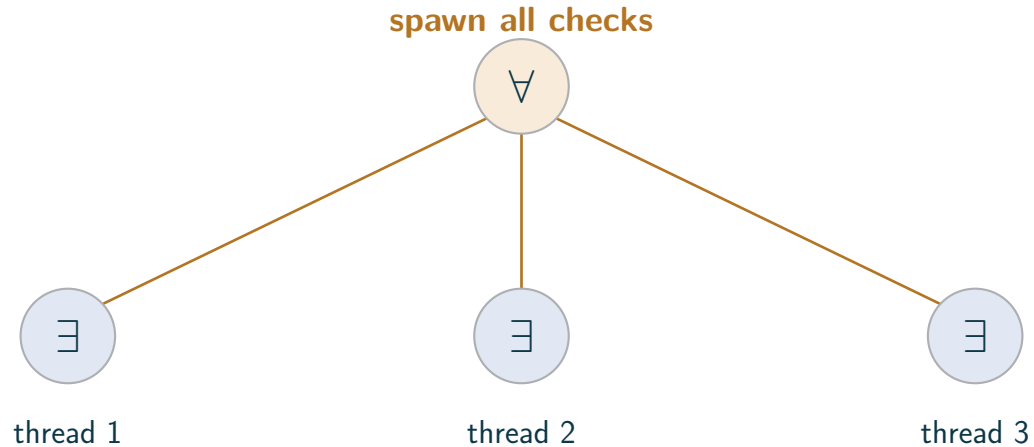
accept if *some* choice works

UNIVERSAL STATE

alternation = “run many threads in parallel”

one thread per triggered witness obligation

accept if *all* threads work



Fix 2: alternation replaces backtracking

EXISTENTIAL STATE

nondeterminism = “guess a solution”

choose one locally consistent type or witness

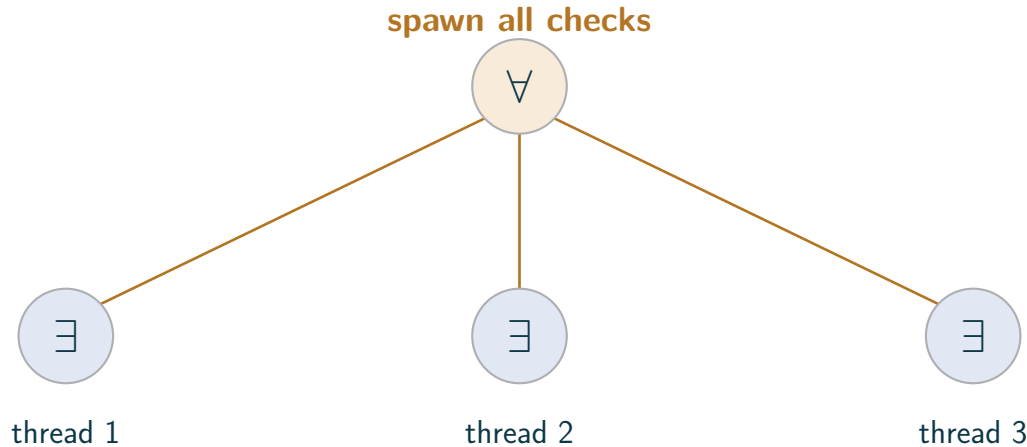
accept if *some* choice works

UNIVERSAL STATE

alternation = “run many threads in parallel”

one thread per triggered witness obligation

accept if *all* threads work



Theorem (Chandra-Kozen-Stockmeyer)

$$\text{APSPACE} = \text{EXPTIME}.$$

Fix 2: alternation replaces backtracking

EXISTENTIAL STATE

nondeterminism = “guess a solution”

choose one locally consistent type or witness

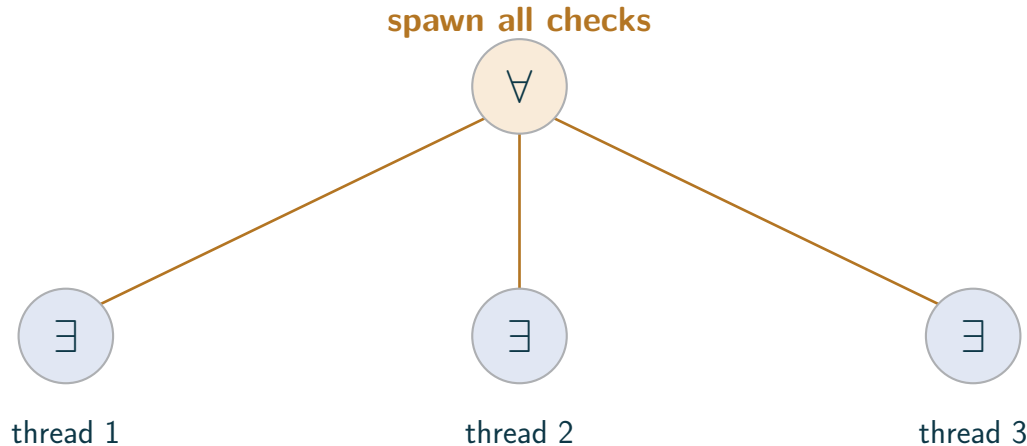
accept if *some* choice works

UNIVERSAL STATE

alternation = “run many threads in parallel”

one thread per triggered witness obligation

accept if *all* threads work



Theorem (Chandra-Kozen-Stockmeyer)

$$\text{APSPACE} = \text{EXPTIME}.$$

An alternating polynomial-space search gives an ExpTime upper bound.

A setup for our algorithm: local configurations

A setup for our algorithm: local configurations

Recall that φ is in normal form: a conjunction of clauses of four shapes.

A setup for our algorithm: local configurations

Recall that φ is in normal form: a conjunction of clauses of **four shapes**.

Our algorithm needs to reflect their **local behavior**.

A setup for our algorithm: local configurations

Recall that φ is in normal form: a conjunction of clauses of **four shapes**.

Our algorithm needs to reflect their **local behavior**.

existence $\exists x (\alpha(x) \wedge \psi(x))$

unary condition $\forall x (\alpha(x) \rightarrow \psi(x))$

pair condition $\forall x \forall y (\beta(x, y) \rightarrow \psi(x, y))$

witness $\forall x (\alpha(x) \rightarrow \exists y (\beta(x, y) \wedge \psi(x, y)))$

A setup for our algorithm: local configurations

Recall that φ is in normal form: a conjunction of clauses of **four shapes**.

Our algorithm needs to reflect their **local behavior**.

existence $\exists x (\alpha(x) \wedge \psi(x))$

unary condition $\forall x (\alpha(x) \rightarrow \psi(x))$

pair condition $\forall x \forall y (\beta(x, y) \rightarrow \psi(x, y))$

witness $\forall x (\alpha(x) \rightarrow \exists y (\beta(x, y) \wedge \psi(x, y)))$

A configuration stores only local types

A setup for our algorithm: local configurations

Recall that φ is in normal form: a conjunction of clauses of **four shapes**.

Our algorithm needs to reflect their **local behavior**.

existence $\exists x (\alpha(x) \wedge \psi(x))$

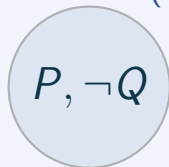
unary condition $\forall x (\alpha(x) \rightarrow \psi(x))$

pair condition $\forall x \forall y (\beta(x, y) \rightarrow \psi(x, y))$

witness $\forall x (\alpha(x) \rightarrow \exists y (\beta(x, y) \wedge \psi(x, y)))$

A configuration stores only local types

1-**TYPE** $t(x)$



a maximal consistent set of
unary literals about one element

A setup for our algorithm: local configurations

Recall that φ is in normal form: a conjunction of clauses of **four shapes**.

Our algorithm needs to reflect their **local behavior**.

existence $\exists x (\alpha(x) \wedge \psi(x))$

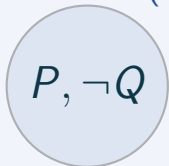
unary condition $\forall x (\alpha(x) \rightarrow \psi(x))$

pair condition $\forall x \forall y (\beta(x, y) \rightarrow \psi(x, y))$

witness $\forall x (\alpha(x) \rightarrow \exists y (\beta(x, y) \wedge \psi(x, y)))$

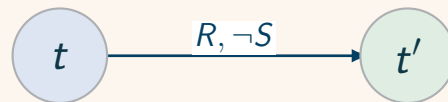
A configuration stores only local types

1-TYPE $t(x)$



a maximal consistent set of
unary literals about one element

GUARDED 2-TYPE $s(x, y)$



the unary endpoint types plus
the atoms of one guarded pair

A setup for our algorithm: local configurations

Recall that φ is in normal form: a conjunction of clauses of **four shapes**.

Our algorithm needs to reflect their **local behavior**.

existence $\exists x (\alpha(x) \wedge \psi(x))$

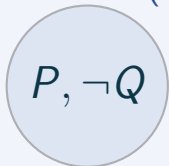
unary condition $\forall x (\alpha(x) \rightarrow \psi(x))$

pair condition $\forall x \forall y (\beta(x, y) \rightarrow \psi(x, y))$

witness $\forall x (\alpha(x) \rightarrow \exists y (\beta(x, y) \wedge \psi(x, y)))$

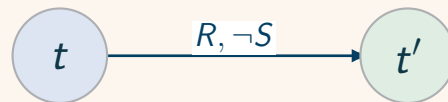
A configuration stores only local types

1-TYPE $t(x)$



a maximal consistent set of
unary literals about one element

GUARDED 2-TYPE $s(x, y)$



the unary endpoint types plus
the atoms of one guarded pair

There are exponentially many possible types, but each type has a **polynomial-size description**.

The alternating local search

The alternating local search

ALGORITHM

- 1 $N \leftarrow 0$.
- 2 Guess the 1-type of a root v .
- 3 If $N > 2^{|\varphi|}$, **accept**.
- 4 Guess at most $|\varphi|$ children of v , their 1-types, and the guarded 2-type between v and each child.
- 5 Check if no chosen type violates universal constraints of φ ; otherwise **reject**.
- 6 Verify that v has all its existential requirements fulfilled; otherwise **reject**.
- 7 Set $N \leftarrow N + 1$.
- 8 Universally choose a child of v as the new v and return to Step 3.



The alternating local search

ALGORITHM

- 1 $N \leftarrow 0$.
- 2 Guess the 1-type of a root v .
- 3 If $N > 2^{|\varphi|}$, **accept**.
- 4 Guess at most $|\varphi|$ children of v , their 1-types, and the guarded 2-type between v and each child.
- 5 Check if no chosen type violates universal constraints of φ ; otherwise **reject**.
- 6 Verify that v has all its existential requirements fulfilled; otherwise **reject**.
- 7 Set $N \leftarrow N + 1$.
- 8 Universally choose a child of v as the new v and return to Step 3.

repeat

WHY IT WORKS

Soundness

An accepting run unfolds into a tree-shaped structure. Steps 5–6 enforce all local conditions; Step 8 checks every chosen subtree.

Completeness

From a tree-shaped model $\mathfrak{A}$, keep one witness child per existential requirement. At most $|\varphi|$ children are needed. Each guess as in $\mathfrak{A}$.

Depth bound

Beyond $2^{|\varphi|}$ levels, a 1-type repeats. Its earlier local behaviour can then be reused.

accepting run $\iff$ tree-shaped model

The alternating local search

ALGORITHM

- 1 $N \leftarrow 0$.
- 2 Guess the 1-type of a root v .
- 3 If $N > 2^{|\varphi|}$, **accept**.
- 4 Guess at most $|\varphi|$ children of v , their 1-types, and the guarded 2-type between v and each child.
- 5 Check if no chosen type violates universal constraints of φ ; otherwise **reject**.
- 6 Verify that v has all its existential requirements fulfilled; otherwise **reject**.
- 7 Set $N \leftarrow N + 1$.
- 8 Universally choose a child of v as the new v and return to Step 3.

repeat

WHY IT WORKS

Soundness

An accepting run unfolds into a tree-shaped structure. Steps 5–6 enforce all local conditions; Step 8 checks every chosen subtree.

Completeness

From a tree-shaped model $\mathfrak{A}$, keep one witness child per existential requirement. At most $|\varphi|$ children are needed. Each guess as in $\mathfrak{A}$.

Depth bound

Beyond $2^{|\varphi|}$ levels, a 1-type repeats. Its earlier local behaviour can then be reused.

accepting run $\iff$ tree-shaped model

Theorem. The satisfiability problem for GF^2 is decidable in $\text{APSPACE} = \text{EXPTIME}$.

Just an idea: ExpTime-hardness of GF^2 via reduction from APSpace-TMs

Just an idea: ExpTime-hardness of GF^2 via reduction from APSpace-TMs

One element x stores an entire $p(n)$ -cell configuration of a TM M .

Just an idea: ExpTime-hardness of GF^2 via reduction from APSpace-TMs

One element x stores an entire $p(n)$ -cell configuration of a TM M .

Polynomially many unary predicates encode its state, head, complete tape, and clock.

Just an idea: ExpTime-hardness of GF^2 via reduction from APSpace-TMs

One element x stores an entire $p(n)$ -cell configuration of a TM M .

Polynomially many unary predicates encode its state, head, complete tape, and clock.

one element x = the full tape

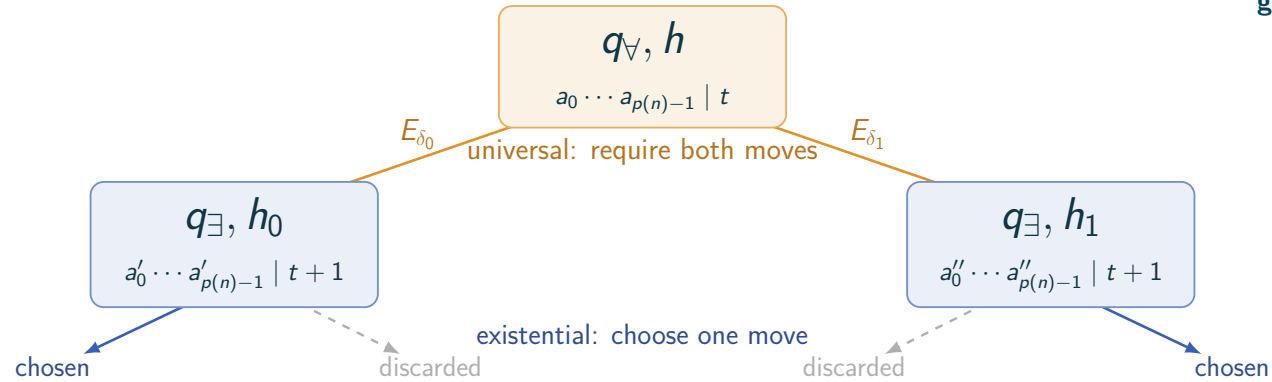
$Q_q(x)$: state q

$H_i(x)$: head at cell i

$A_{i,a}(x)$: cell i contains a

$B_j(x)$: clock bit j

$O(p(n)|\Gamma|)$ predicates



guarded edge = one step

$E_{\delta}(x, y)$ links
configuration x to
its δ -successor y

Just an idea: ExpTime-hardness of GF^2 via reduction from APSpace-TMs

One element x stores an entire $p(n)$ -cell configuration of a TM M .

Polynomially many unary predicates encode its state, head, complete tape, and clock.

one element x = the full tape

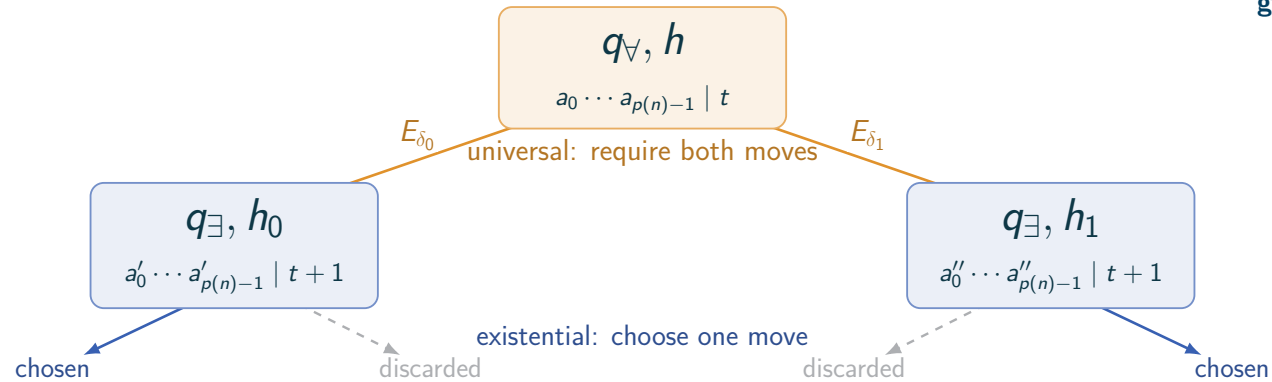
$Q_q(x)$: state q

$H_i(x)$: head at cell i

$A_{i,a}(x)$: cell i contains a

$B_j(x)$: clock bit j

$O(p(n)|\Gamma|)$ predicates



guarded edge = one step

$E_{\delta}(x, y)$ links configuration x to its δ -successor y

existential state

$$\forall x \left(Q_q x \rightarrow \bigvee_{\delta \in \Delta(q)} \exists y (E_{\delta} xy \wedge \text{Step}_{\delta}(x, y)) \right)$$

$\text{Step}_{\delta}(x, y)$ copies all tape labels from x to y , updates the scanned cell and head, and increments the clock; maximal time requires acceptance.

universal state

$$\forall x \left(Q_q x \rightarrow \bigwedge_{\delta \in \Delta(q)} \exists y (E_{\delta} xy \wedge \text{Step}_{\delta}(x, y)) \right)$$

Just an idea: ExpTime-hardness of GF^2 via reduction from APSpace-TMs

One element x stores an entire $p(n)$ -cell configuration of a TM M .

Polynomially many unary predicates encode its state, head, complete tape, and clock.

one element x = the full tape

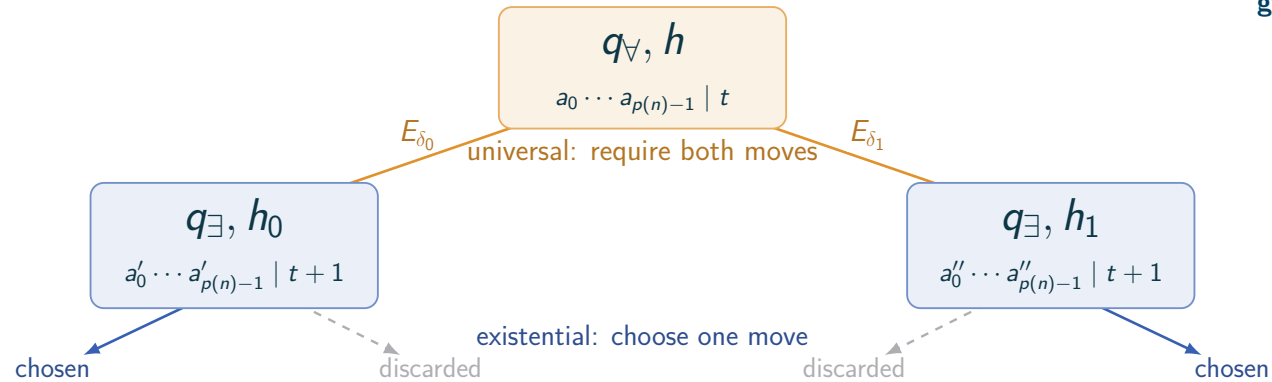
$Q_q(x)$: state q

$H_i(x)$: head at cell i

$A_{i,a}(x)$: cell i contains a

$B_j(x)$: clock bit j

$O(p(n)|\Gamma|)$ predicates



guarded edge = one step

$E_{\delta}(x, y)$ links configuration x to its δ -successor y

existential state

$$\forall x \left(Q_q x \rightarrow \bigvee_{\delta \in \Delta(q)} \exists y (E_{\delta} xy \wedge \text{Step}_{\delta}(x, y)) \right)$$

$\text{Step}_{\delta}(x, y)$ copies all tape labels from x to y , updates the scanned cell and head, and increments the clock; maximal time requires acceptance.

universal state

$$\forall x \left(Q_q x \rightarrow \bigwedge_{\delta \in \Delta(q)} \exists y (E_{\delta} xy \wedge \text{Step}_{\delta}(x, y)) \right)$$

M accepts $w \iff \varphi_{M,w}$ is satisfiable; hence $\text{SAT}(GF^2)$ is APSpace-hard = ExpTime-hard.

Take-away messages

Take-away messages

GF² IN FIVE LINES

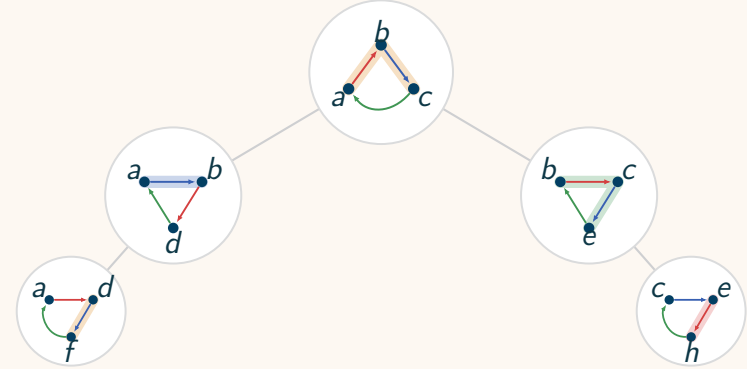
- In GF², every quantifier is controlled by an atom covering the relevant variables.
- Guards enforce **locality**: non-adjacent elements cannot be compared.
- Unravelling turns arbitrary models into tree-shaped models with the same guarded theory.
- Repeated 1-types bound branch depth; alternation checks all witness obligations.
- Each configuration is polynomial-size; therefore satisfiability is in EXPTIME.

Take-away messages

GF^2 IN FIVE LINES

- In GF^2 , every quantifier is controlled by an atom covering the relevant variables.
- Guards enforce **locality**: non-adjacent elements cannot be compared.
- Unravelling turns arbitrary models into tree-shaped models with the same guarded theory.
- Repeated 1-types bound branch depth; alternation checks all witness obligations.
- Each configuration is polynomial-size; therefore satisfiability is in $EXPTIME$.

FULL GF : LINEARLY BOUNDED TREEWIDTH



bands mark guarded tuples ■ repeated elements stay connected

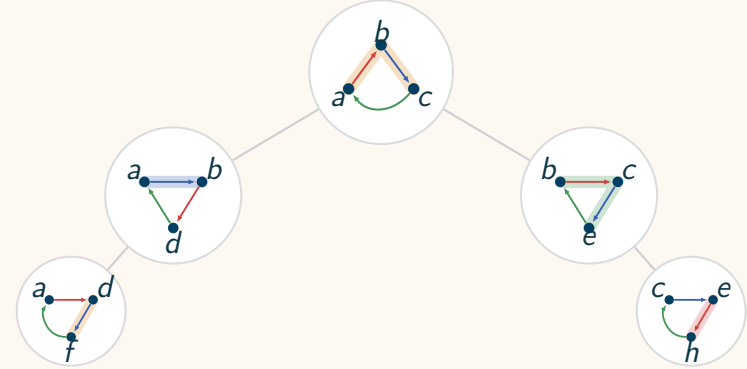
bag size $\leq \text{width}(\varphi)$, hence $\text{tw} = O(|\varphi|)$

Take-away messages

GF² IN FIVE LINES

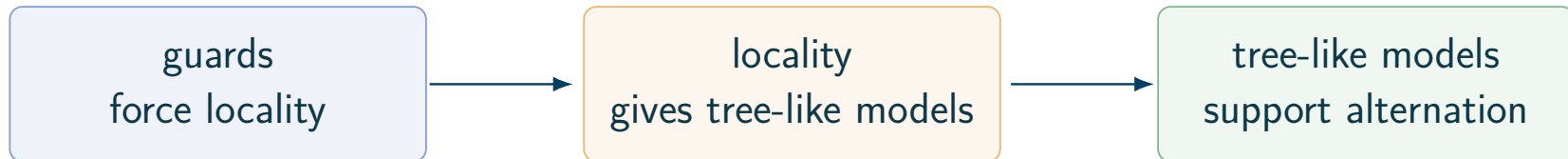
- In GF², every quantifier is controlled by an atom covering the relevant variables.
- Guards enforce **locality**: non-adjacent elements cannot be compared.
- Unravelling turns arbitrary models into tree-shaped models with the same guarded theory.
- Repeated 1-types bound branch depth; alternation checks all witness obligations.
- Each configuration is polynomial-size; therefore satisfiability is in EXPTIME.

FULL GF: LINEARLY BOUNDED TREEWIDTH



bag size $\leq \text{width}(\varphi)$, hence $\text{tw} = O(|\varphi|)$

THE GUARDED RECIPE SCALES BEYOND TWO VARIABLES

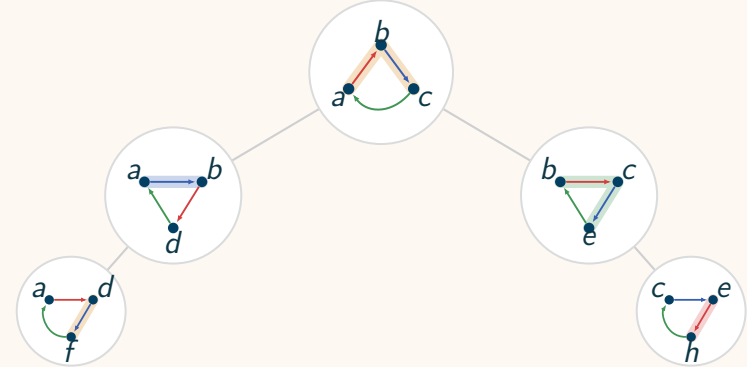


Take-away messages

GF² IN FIVE LINES

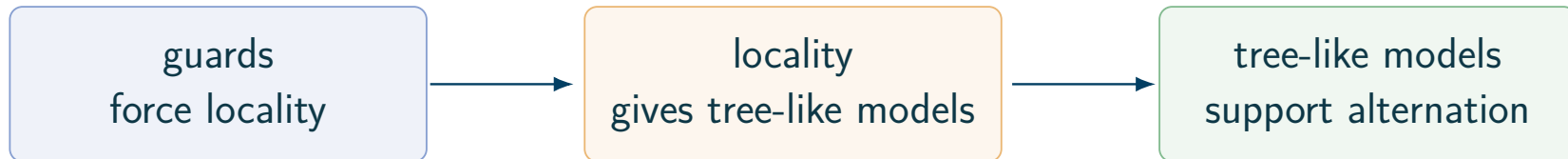
- In GF², every quantifier is controlled by an atom covering the relevant variables.
- Guards enforce **locality**: non-adjacent elements cannot be compared.
- Unravelling turns arbitrary models into tree-shaped models with the same guarded theory.
- Repeated 1-types bound branch depth; alternation checks all witness obligations.
- Each configuration is polynomial-size; therefore satisfiability is in EXPTIME.

FULL GF: LINEARLY BOUNDED TREEWIDTH



bag size $\leq \text{width}(\varphi)$, hence $\text{tw} = O(|\varphi|)$

THE GUARDED RECIPE SCALES BEYOND TWO VARIABLES



Full GF with unbounded arity is 2EXPTIME-complete ■ Full GF has the finite model property.

